



Diplomado en Data Science Aplicada con Python para la Toma de Decisiones

Clase 2: Control de Flujo — Condicionales, Listas y Ciclos



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Recap Clase 1 + Temas de Hoy

✓ Ya sabemos hacer:

- ✓ `print()` y operaciones aritméticas
- ✓ Variables y nombres (`snake_case`)
- ✓ Tipos (`int`, `float`, `str`, `bool`)
- ✓ Conversiones `int()`, `float()`, `str()`
- ✓ Leer datos con `input()`
- ✓ `if` / `elif` / `else` básico

▶▶ Hoy vamos a profundizar en:

- **Condicionales a fondo:** cuándo usar `if` solo, `if/else`, `if/elif/else` y las trampas comunes
- **Listas con profundidad:** slicing, métodos, operador `in`
- **Ciclos for aplicados:** `range`, `enumerate`, `zip` y los patrones más útiles

Al final: reporte de una batería completa de pozos.

Condicionales a Fondo

Cómo el programa toma decisiones

Anatomía de if

Un `if` evalúa una condición. Si es `True`, ejecuta el bloque; si es `False`, lo salta.

Python

```
presion = 3200

if presion > 3000:
    print("Alerta: presion alta")

print("Fin")
```

>_ Output

```
Alerta: presion alta
Fin
```

- `if` + condición + :
- El **bloque** son las líneas con **4 espacios** de indentación.
- `print("Fin")` **no** está indentado: se ejecuta **siempre**.

if Solo — “Hacer Algo Solo Si...”

Se usa cuando no hay caso alternativo: solo actuamos si pasa algo especial.

Python

```
presion = 2800

if presion > 3000:
    print("Alerta: presion alta")
```

>_ Output

(no imprime nada)

Cuándo usarlo:

- ✓ Enviar una alerta cuando algo falla.
- ✓ Registrar un evento cuando ocurre.
- ✓ Aplicar un descuento solo si el pedido supera cierto monto.

Como la presión es 2800, la condición es False y no pasa nada. No hace falta else.

if / else — Dos Caminos Exclusivos

Se usa cuando siempre hacemos algo: la pregunta es *cuál* de las dos cosas.

 Python

```
bopd = 750

if bopd >= 500:
    estado = "activo"
else:
    estado = "cerrado"

print("Pozo:", estado)
```

- Se ejecuta **exactamente una** de las dos ramas. Nunca ambas, nunca ninguna.
- **else** **no** lleva condición: es “todos los demás casos”.
- Después del bloque, estado siempre tiene un valor.

>_ Output

Pozo: activo

if / elif / else — Cadena de Casos

Varios caminos posibles. Python evalúa de arriba abajo y ejecuta solo el primero que sea True.

Python

```
bopd = 1250

if bopd >= 1000:
    nivel = "alto"
elif bopd >= 300:
    nivel = "medio"
else:
    nivel = "workover"

print(nivel)
```

- elif = "else if". Se repite tantas veces como haga falta.
- bopd >= 1000 es True → asigna "alto" y **sale** de la cadena.
- El elif de abajo ni se prueba, aunque también sería True.

>_ Output

alto

Trampa: `if` / `if` / `if` $\neq$ `if` / `elif` / `elif`

Tres `if` sueltos son independientes. Una cadena `if` / `elif` es una sola decisión.

✗ Antipatrón: `if` sueltos

Python

```
bopd = 1500

if bopd >= 300:
    print("medio")
if bopd >= 800:
    print("alto")
if bopd >= 1200:
    print("muy alto")
```

>_ Output

```
medio
alto
muy alto
```

✓ Correcto: cadena `elif`

Python

```
bopd = 1500

if bopd >= 1200:
    print("muy alto")
elif bopd >= 800:
    print("alto")
elif bopd >= 300:
    print("medio")
```

>_ Output

```
muy alto
```


else como "Catch-all"

else atrapa todo lo que no capturaron los casos anteriores.

✓ Útil: garantizar un valor

Python

```
if bopd >= 1000:
    nivel = "alto"
elif bopd >= 300:
    nivel = "medio"
else:
    nivel = "revisar"
```

Sin else, si ningún caso aplica, nivel nunca se crea y el print(nivel) de después falla.

⚠ Riesgo: esconder errores

Python

```
tipo = "geolgico"

if tipo == "geologico":
    accion = "revisar corte"
else:
    accion = "nada"
```

*El error de tipeo cae al else en silencio. A veces conviene **no** poner else y que Python avise.*

Combinar Condiciones: and, or

Cuando la decisión depende de más de una condición. (corte = corte de agua.)

and — deben cumplirse las dos

Python

```
bopd = 1250
corte = 0.32

if bopd >= 1000 and corte <
    ↪ 0.5:
    print("optimo")
```

>_ Output

optimo

or — basta con una

Python

```
bopd = 180
corte = 0.75

if bopd < 200 or corte > 0.9:
    print("candidato a cierre")
```

>_ Output

candidato a cierre

Negar con not y Agrupar con Paréntesis

not invierte True/False

Python

```
activo = False

if not activo:
    print("pozo inactivo")
```

>_ Output

```
pozo inactivo
```

not True → False not False → True

Paréntesis para agrupar

Python

```
bopd = 1200
corte = 0.55
presion = 3100

if (bopd >= 1000 and corte < 0.5)
    ↪ \
    or presion > 3000:
    print("alerta operativa")
```

>_ Output

```
alerta operativa
```

Práctica Guiada: Alertas de Pozo

🕒 6 min

Del pozo Sacha-042 llegan cuatro mediciones:

Python

```
bopd = 1250  
corte = 0.32  
presion = 3200  
temperatura = 95
```

Escriban las alertas (son **independientes**, pueden dispararse varias):

- ! $\text{presion} > 3000 \rightarrow$ “**ALERTA: presion alta**”
- ! $\text{corte} > 0.7 \rightarrow$ “**AVISO: alto corte de agua**”
- ! $\text{bopd} < 300 \text{ and } \text{corte} > 0.5 \rightarrow$ “**CRITICO: cierre**”
- ! $\text{temperatura} > 100 \rightarrow$ “**AVISO: temperatura alta**”

Solución — Alertas de Pozo

Python

```
bopd = 1250
corte = 0.32
presion = 3200
temperatura = 95

if presion > 3000:
    print("ALERTA: presion alta")

if corte > 0.7:
    print("AVISO: alto corte de agua")

if bopd < 300 and corte > 0.5:
    print("CRITICO: cierre")

if temperatura > 100:
    print("AVISO: temperatura alta")
```

>_ Output

ALERTA: presion alta

- ✓ Son if **sueltos**: las alertas son independientes.
- ✓ Con estos datos solo se dispara una, pero otro pozo podría disparar dos o tres.

Su Turno: Semáforo de Producción

🕒 6 min

Clasificar cada pozo con un solo color según su producción.

Reglas (mutuamente excluyentes):

- **VERDE:** bopd $\geq$ 1000
- **AMARILLO:** entre 500 y 999
- **ROJO:** menos de 500

Python

```
bopd = int(input("BOPD del pozo: "))
```

💡 Un pozo tiene **un** color → es cadena if/elif/else, no if sueltos.

Solución — Semáforo de Producción

Python

```
bopd = int(input("BOPD del pozo: "))

if bopd >= 1000:
    color = "VERDE"
elif bopd >= 500:
    color = "AMARILLO"
else:
    color = "ROJO"

print("Semaforo:", color)
```

- ✓ Cadena if/elif/else: corre solo una rama.
- ✓ No hace falta escribir `bopd < 500`: si no fue ≥ 1000 ni ≥ 500 , el else lo atrapa.

Listas

Cuando el dato no es uno, son muchos

¿Qué es una Lista?

En la Clase 1 teníamos un pozo. Hoy el supervisor pide la producción de los 5 pozos de la batería.

✗ Sin listas: una variable por pozo

Python

```
bopd_1 = 1250
bopd_2 = 780
bopd_3 = 340
bopd_4 = 1600
bopd_5 = 950
```

5 pozos, 5 variables. ¿Y si son 500? Imposible de manejar.

✓ Con una lista: un solo nombre

Python

```
bopd = [1250, 780, 340, 1600,
        ↪ 950]
```

- Los corchetes [] delimitan la lista.
- Las comas separan los elementos.
- Es **ordenada**: cada valor tiene una posición fija.

Acceder a un Elemento: el Índice

Cada elemento tiene una posición (índice). Python cuenta desde 0.

Python

```
bopd = [1250, 780, 340, 1600, 950]
```

```
print(bopd[0])    # primer pozo
print(bopd[2])    # tercero
print(bopd[-1])   # ultimo
print(len(bopd))  # cuantos hay
```

índice de cada elemento:

1250	780	340	1600	950
0	1	2	3	4

- El primer elemento es [0], no [1].
- [-1] es el último (cuenta desde el final).
- len() dice cuántos elementos hay.

>_ Output

```
1250
340
950
5
```

Slicing: Cortar un Pedazo de la Lista

`lista[a:b]` devuelve desde el índice `a` hasta `b-1` (incluye `a`, excluye `b`).

Python

```
prod = [1250, 1180, 1220, 1150,
        ↪ 1300]

print(prod[0:3])    # índices 0,1,2
print(prod[2:])     # del 2 al final
print(prod[:2])     # los 2 primeros
print(prod[-2:])    # los 2 últimos
```

>_ Output

```
[1250, 1180, 1220]
[1220, 1150, 1300]
[1250, 1180]
[1150, 1300]
```

Omitir el primer número = "desde el inicio". Omitir el segundo = "hasta el final".

Modificar una Lista: los Métodos

Un método es una acción que se llama con la sintaxis `lista.metodo(...)`.

Python

```
pozos = ["Sacha-042", "Sacha-108"]

pozos.append("Auca-63")
pozos.insert(0, "Auca-91")
pozos.remove("Sacha-108")

print(pozos)
```

>_ Output

```
['Auca-91', 'Sacha-042',  
'Auca-63']
```

Método

Qué hace

<code>.append(x)</code>	agrega x al final
<code>.insert(i,x)</code>	inserta en posición i
<code>.remove(x)</code>	borra la primera x
<code>.pop()</code>	quita y devuelve el último

Operador in: ¿Está el Valor en la Lista?

valor in lista devuelve True si aparece; False si no.

Python

```
ruta = ["Sacha-042", "Auca-63"]

print("Sacha-042" in ruta)
print("Auca-91" in ruta)

if "Sacha-042" in ruta:
    print("visitar hoy")
```

>_ Output

```
True
False
visitar hoy
```

- Sirve para **decidir** sin escribir un ciclo.
- Combina muy bien con if: "si este pozo está en la ruta..."
- También funciona en texto: "gas" in "gas condensado".

Listas Paralelas: Una Tabla Sin Librerías

Dos o más listas donde la posición conecta la información.

Python

```
pozos = ["Sacha-042", "Sacha-108"]  
bopd = [1250, 780]  
corte = [0.32, 0.55]  
  
# posicion 0 = mismo pozo  
print(pozos[0], bopd[0], corte[0])
```

>_ Output

```
Sacha-042 1250 0.32
```

- Regla de oro: **todas las listas deben tener el mismo largo.**
- Es la forma más simple de manejar una tabla de datos.
- ✓ En la Clase 3 esto se convierte en un DataFrame de pandas.

Práctica: Producción Semanal

🕒 7 min

Producción diaria del pozo Sacha-042 en los últimos 7 días:

Python

```
prod = [1250, 1180, 1220, 1150, 1300, 1280, 1240]
```

Sin usar for todavía, con lo que ya sabemos:

1. Imprimir la producción del **lunes** ([0]) y del **domingo** ([-1]).
2. Imprimir **lunes a miércoles** con un slice.
3. Agregar un octavo día con **1200** usando .append().
4. Preguntar si **1300** está en la lista (in).
5. Imprimir cuántos días hay ahora (len()).

Solución — Producción Semanal

Python

```
prod = [1250, 1180, 1220, 1150,  
        1300, 1280, 1240]  
  
print(prod[0], prod[-1])  
print(prod[0:3])  
  
prod.append(1200)  
  
print(1300 in prod)  
print(len(prod))
```

>_ Output

```
1250 1240  
[1250, 1180, 1220]  
True  
8
```

- ✓ `.append()` pasó la lista de 7 a 8.
- ✓ `len()` lo confirma.

Ciclos for

Aplicar la misma lógica a muchos datos

Anatomía del for

for recorre una lista: para cada elemento ejecuta el bloque.

Python

```
bopd = [1250, 780, 1600]

for prod in bopd:
    print(prod, "BOPD")

print("fin")
```

- prod es la **variable de bucle**: cambia en cada vuelta.
- in bopd le dice a Python **qué recorrer**.
- Solo lo **indentado** se repite; print("fin") corre al final.

>_ Output

```
1250 BOPD
780 BOPD
1600 BOPD
fin
```

range(n): Números para Contar

range(n) genera 0, 1, ..., n-1. Útil para repetir un número fijo de veces.

Python

```
for dia in range(7):  
    print("dia", dia + 1)
```

>_ Output

```
dia 1  
dia 2  
...  
dia 7
```

Python

```
for h in range(6, 12):  
    print("hora", h)
```

>_ Output

```
hora 6  
hora 7  
...  
hora 11
```

💡 range **no incluye** el número final. range(6, 12) llega hasta 11, no 12.

enumerate(): Índice y Valor a la Vez

Cuando además del valor necesitamos su posición (pozo 1, pozo 2...).

 Python

```
pozos = ["Sacha-042", "Auca-63"]  
  
for i, pozo in enumerate(pozos):  
    print(i, "->", pozo)
```

>_ Output

```
0 -> Sacha-042  
1 -> Auca-63
```

- i, pozo son **dos variables** a la vez: i la posición, pozo el elemento.
- Arranca en 0. Para empezar en 1: `enumerate(pozos, 1)`.

zip(): Recorrer Dos Listas en Paralelo

`zip(a, b)` empareja los elementos de `a` y `b` posición por posición.

Python

```
pozos = ["Sacha-042", "Sacha-108"]  
bopd  = [1250, 780]  
  
for pozo, prod in zip(pozos, bopd):  
    print(pozo, prod)
```

>_ Output

```
Sacha-042 1250  
Sacha-108 780
```

- `pozo, prod`: otra vez dos variables, una por lista.
- ✓ Es la forma limpia de recorrer **listas paralelas**.

Patrón Acumulador: Sumar Todo

Empezamos en `total = 0` y en cada vuelta sumamos el elemento actual.

Python

```
prod = [1250, 1180, 1220, 1150]

total = 0
for p in prod:
    total = total + p

print("Total:", total)
```

- `total = 0` va **antes** del ciclo (el estado inicial).
- `total = total + p` actualiza en cada vuelta.
- ✓ Atajo: `sum(prod)` hace lo mismo en una línea.

>_ Output

```
Total: 4800
```

Patrón Contador: ¿Cuántos Cumplen?

Empezamos en $n = 0$ y sumamos 1 cada vez que se cumple una condición.

Python

```
bopd = [1250, 780, 1600, 340, 920]

n = 0
for prod in bopd:
    if prod >= 1000:
        n = n + 1

print("Sobre cutoff:", n)
```

>_ Output

```
Sobre cutoff: 2
```

- Un if **dentro** del for: revisamos cada pozo por separado.
- Solo sumamos al contador cuando la condición es True.

Patrón Filtro: Guardar los que Cumplen

Empezamos con una lista vacía y hacemos `.append()` de los que pasan el filtro.

Python

```
pozos = ["Sacha-042", "Auca-63",  
         "Shushufindi-21"]  
bopd  = [1250, 340, 1600]  
  
altos = []  
for pozo, prod in zip(pozos, bopd):  
    if prod >= 1000:  
        altos.append(pozo)  
  
print(altos)
```

>_ Output

```
['Sacha-042',  
 'Shushufindi-21']
```

altos = [] crea la lista vacía. El for recorre, el if filtra y .append() guarda.

Práctica: Reporte de una Batería

🕒 15 min

Cinco pozos de un campo en la Amazonía. Generemos el reporte del día.

🔗 Python

```
pozos = ["Sacha-042", "Sacha-108", "Auca-63", "Shushufindi-21",  
↪ "Auca-91"]  
bopd = [1250, 780, 340, 1600, 950]  
corte = [0.32, 0.55, 0.78, 0.25, 0.48]
```

Con lo de hoy, calculen e impriman:

1. **Producción total** de la batería (acumulador).
2. **Cuántos pozos** son rentables, `bopd >= 500` (contador).
3. **Lista de pozos** con `corte > 0.5` (filtro).
4. **Semáforo** de cada pozo con `if/elif/else`.

Solución (1/2): Total y Rentables

Python

```
bopd = [1250, 780, 340, 1600, 950]

total = 0
for prod in bopd:
    total = total + prod
print("Total:", total, "BOPD")

rentables = 0
for prod in bopd:
    if prod >= 500:
        rentables = rentables + 1
print("Rentables:", rentables)
```

>_ Output

```
Total: 4920 BOPD
Rentables: 4
```

- ✓ Parte 1: patrón **acumulador** para sumar toda la producción.
- ✓ Parte 2: patrón **contador** para contar los rentables.

Solución (2/2): Filtro y Semáforo

Usando las mismas listas pozos, bopd y corte de la práctica.

Python

```
con_agua = []
for pozo, agua in zip(pozos, corte):
    if agua > 0.5:
        con_agua.append(pozo)
print("Alto corte:", con_agua)

for pozo, prod in zip(pozos, bopd):
    if prod >= 1000:
        color = "VERDE"
    elif prod >= 500:
        color = "AMARILLO"
    else:
        color = "ROJO"
    print(pozo, color)
```

>_ Output

```
Alto corte: ['Sacha-108',
             'Auca-63']
Sacha-042 VERDE
Sacha-108 AMARILLO
Auca-63 ROJO
Shushufindi-21 VERDE
Auca-91 AMARILLO
```

Parte 3: **filtro**. Parte 4: **semáforo** con if/elif/else.

Cierre

Recapitulación y próxima clase

Lo Que Aprendimos Hoy

Condicionales

- ✓ if solo, if/else, if/elif/else
- ✓ if/if/if **no** es if/elif
- ✓ else como default (y cuándo omitirlo)
- ✓ and, or, not, paréntesis

Listas

- ✓ Qué es, sintaxis [], índice desde 0
- ✓ Slicing [a:b]
- ✓ .append, .insert, .remove, .pop
- ✓ Operador in, listas paralelas

Ciclos for

- ✓ Anatomía e indentación
- ✓ range() para contar
- ✓ enumerate() (índice + valor)
- ✓ zip() (dos listas paralelas)
- ✓ Patrones: acumulador, contador, filtro

Habilidad clave *Producir el reporte diario de una batería completa de pozos, combinando listas, for e if.*

Próxima Clase: Del for al DataFrame

Hoy recorrimos 5 pozos con `for`. ¿Y si fueran 30 años de producción diaria?

- ➔ **pandas** — DataFrames: las listas paralelas de hoy con **superpoderes** (filtrar, agrupar y sumar en una línea).
- ➔ **Lectura de datos reales** — archivos `.csv`, `.xlsx` y `.json` de producción y sensores.
- ➔ **Well logs LAS** — leer registros petrofísicos con `lasio`.
- ➔ **Limpieza de datos** — valores faltantes y normalización, con una práctica sobre 30 años de un campo real.

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Diplomado en Data Science Aplicada con Python

SLB Ecuador · UDLA · 2026